

Chapter 7

Grover's Search Algorithm

The second-most important quantum algorithm after Shor's is Grover's search algorithm [125]. It doesn't provide an exponential speed-up, only a quadratic speed-up, but it is much more widely applicable than Shor.

7.1 The problem

The search problem:

For $N = 2^n$, we are given an arbitrary $x \in \{0, 1\}^N$. The goal is to find an i such that $x_i = 1$ (and to output 'no solutions' if there are no such i). We denote the number of solutions in x by t (i.e., t is the Hamming weight of x).

This problem may be viewed as a simplification of the problem of searching an N -slot unordered database or search space, modeled by an N -bit string. Classically, a randomized algorithm would need $\Theta(N)$ queries to solve the search problem. Grover's algorithm solves it in $O(\sqrt{N})$ queries, and $O(\sqrt{N} \log N)$ other gates (the number of gates can be reduced a bit further, see Exercise 9).

7.2 Grover's algorithm

Let $O_{x,\pm}|i\rangle = (-1)^{x_i}|i\rangle$ denote the $\pm$ -type oracle for the input x (i.e., a phase-query), and R_0 be the unitary transformation that puts a -1 in front of all basis states $|i\rangle$ where $i \neq 0^n$, and that does nothing to the basis state $|0^n\rangle$.¹ The *Grover iterate* is

$$\mathcal{G} = H^{\otimes n} R_0 H^{\otimes n} O_{x,\pm}. \quad (7.1)$$

Note that 1 Grover iterate makes 1 query, and uses $O(\log N)$ other gates.

Grover's algorithm starts in the n -bit state $|0^n\rangle$, applies a Hadamard transformation to each qubit to get the uniform superposition $|U\rangle = \frac{1}{\sqrt{N}} \sum_i |i\rangle$ of all N indices, applies $\mathcal{G}$ to this state k times (for some k to be chosen later), and then measures the final state. Intuitively, what happens is that in each iteration some amplitude is moved from the indices of the 0-bits to the indices of the 1-bits. The algorithm stops when almost all of the amplitude is on the 1-bits, in which case a measurement of the final state will probably give the index of a 1-bit. Figure 7.1 illustrates this.

¹This unitary R_0 is independent of x , and can be implemented using $O(n)$ elementary gates (Exercise 2.8.a).

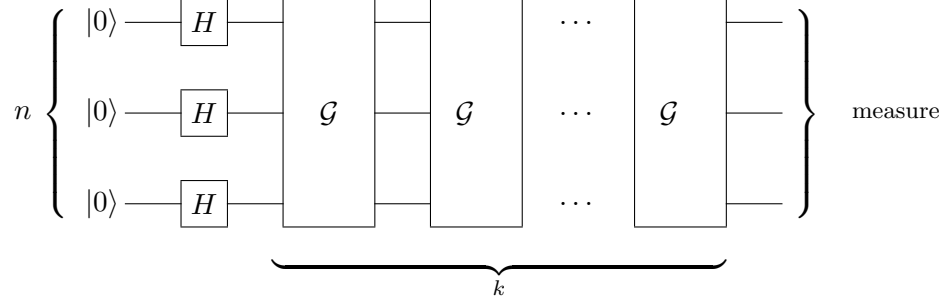


Figure 7.1: Grover's algorithm, with k Grover iterates

In order to analyze this, define the following “good” and “bad” states, corresponding to the solutions and non-solutions, respectively:

$$|G\rangle = \frac{1}{\sqrt{t}} \sum_{i:x_i=1} |i\rangle \text{ and } |B\rangle = \frac{1}{\sqrt{N-t}} \sum_{i:x_i=0} |i\rangle.$$

Then the uniform state over all indices can be written as

$$|U\rangle = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |i\rangle = \sin(\theta)|G\rangle + \cos(\theta)|B\rangle, \quad \text{for } \theta = \arcsin(\sqrt{t/N}).$$

The Grover iterate $\mathcal{G}$ is actually the product of two *reflections*.² Firstly, $O_{x,\pm}$ is a reflection through the subspace V spanned by the basis states that are not solutions; restricted to the 2-dimensional space spanned by $|G\rangle$ and $|B\rangle$ this is in fact just a reflection through the state $|B\rangle$. Secondly,

$$H^{\otimes n} R_0 H^{\otimes n} = H^{\otimes n} (2|0^n\rangle\langle 0^n| - I) H^{\otimes n} = 2H^{\otimes n} |0^n\rangle\langle 0^n| H^{\otimes n} - H^{\otimes n} I H^{\otimes n} = 2|U\rangle\langle U| - I$$

is a reflection through $|U\rangle$.

Here is Grover's algorithm restated, assuming we know the fraction of solutions is $\varepsilon = t/N$:

1. Set up the starting state $|U\rangle = H^{\otimes n}|0\rangle$
2. Repeat the following $k = O(1/\sqrt{\varepsilon})$ times:
 - (a) Reflect through $|B\rangle$ (i.e., apply $O_{x,\pm}$)
 - (b) Reflect through $|U\rangle$ (i.e., apply $H^{\otimes n} R_0 H^{\otimes n}$)
3. Measure the first register and check that the resulting i is a solution

²A reflection through a subspace V is a unitary A such that $Av = v$ for all vectors $v \in V$, and $Aw = -w$ for all w orthogonal to V . Note that we can write $A = 2P_V - I$, where P_V is the projector onto subspace V . If V is 1-dimensional, spanned by vector u , we also call this a reflection through u .

Geometric argument: There is a fairly simple geometric argument why the algorithm works. The analysis is in the 2-dimensional real plane spanned by $|B\rangle$ and $|G\rangle$. We start with

$$|U\rangle = \sin(\theta)|G\rangle + \cos(\theta)|B\rangle.$$

The two reflections (a) and (b) increase the angle from θ to 3θ , moving us towards the good state as illustrated in Figure 7.2.

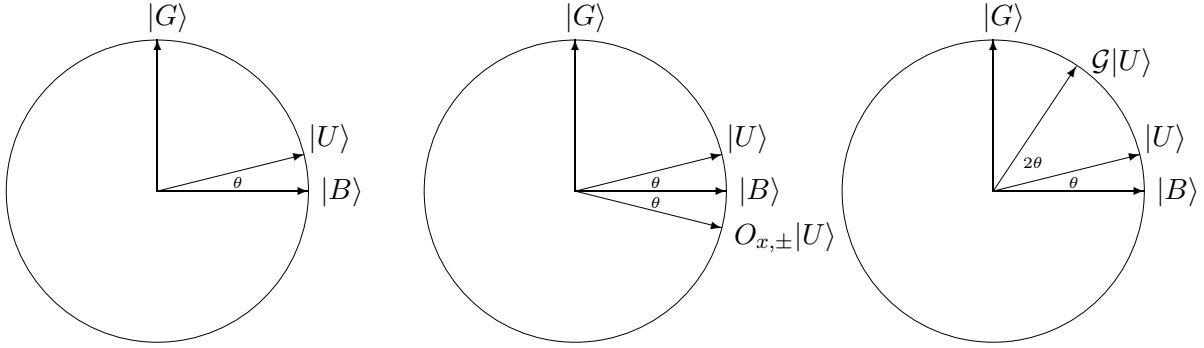


Figure 7.2: The first iteration of Grover: (picture on the left) start with $|U\rangle$; (middle) reflect through $|B\rangle$ to get $O_{x,\pm}|U\rangle$; (right) reflect through $|U\rangle$ to get $\mathcal{G}|U\rangle$

The next two reflections (a) and (b) increase the angle with another 2θ , etc. More generally, after k applications of (a) and (b) our state has become

$$\sin((2k+1)\theta)|G\rangle + \cos((2k+1)\theta)|B\rangle.$$

If we now measure, the probability of seeing a solution is $P_k = \sin^2((2k+1)\theta)$. We want P_k to be as close to 1 as possible. Note that if we can choose $\tilde{k} = \frac{\pi}{4\theta} - 1/2$, then $(2\tilde{k}+1)\theta = \pi/2$ and hence $P_{\tilde{k}} = \sin^2(\pi/2) = 1$. An example where this works is if $t = N/4$, for then $\theta = \pi/6$ and $\tilde{k} = 1$.

Unfortunately $\tilde{k} = \frac{\pi}{4\theta} - 1/2$ will usually not be an integer, and we can only do an integer number of Grover iterations. However, if we choose k to be the integer closest to $\tilde{k}$, then our final state will still be close to $|G\rangle$ and the failure probability will still be small (assuming $t \ll N$):

$$\begin{aligned} 1 - P_k &= \cos^2((2k+1)\theta) = \cos^2((2\tilde{k}+1)\theta + 2(k-\tilde{k})\theta) \\ &= \cos^2(\pi/2 + 2(k-\tilde{k})\theta) = \sin^2(2(k-\tilde{k})\theta) \leq \sin^2(\theta) = \frac{t}{N}, \end{aligned}$$

where we used $|k - \tilde{k}| \leq 1/2$. Since $\arcsin(\theta) \geq \theta$, the number of queries is $k \leq \frac{\pi}{4\theta} \leq \frac{\pi}{4} \sqrt{\frac{N}{t}}$.

Algebraic argument: For those who don't like geometry, here's an alternative (but equivalent) algebraic argument. Let a_k denote the amplitude of the indices of the t 1-bits after k Grover iterates, and b_k the amplitude of the indices of the 0-bits. Initially, for the uniform superposition $|U\rangle$ we have $a_0 = b_0 = 1/\sqrt{N}$. Using that $H^{\otimes n} R_0 H^{\otimes n} = \frac{2}{N} J - I$, where J is the $N \times N$ all-1s matrix, we find the following recursion:

$$\begin{aligned} a_{k+1} &= \frac{N-2t}{N} a_k + \frac{2(N-t)}{N} b_k \\ b_{k+1} &= \frac{-2t}{N} a_k + \frac{N-2t}{N} b_k \end{aligned}$$

The following formulas, due to Boyer et al. [62], provide a closed form for a_k and b_k (which may be verified by substituting them into the recursion). With $\theta = \arcsin(\sqrt{t/N})$ as before, define

$$\begin{aligned} a_k &= \frac{1}{\sqrt{t}} \sin((2k+1)\theta) \\ b_k &= \frac{1}{\sqrt{N-t}} \cos((2k+1)\theta) \end{aligned}$$

Accordingly, after k iterations the success probability (the sum of squares of the amplitudes of the locations of the t 1-bits) is the same as in the geometric analysis

$$P_k = t \cdot a_k^2 = (\sin((2k+1)\theta))^2.$$

Thus we have a bounded-error quantum search algorithm with $O(\sqrt{N/t})$ queries, *assuming we know t* . We now list (without full proofs) a number of useful variants of Grover:

- If we know t exactly, then the algorithm can be tweaked to end up in *exactly* the good state. Roughly speaking, you can make the angle θ slightly smaller, such that $\tilde{k} = \pi/4\theta - 1/2$ becomes an integer (see Exercise 5).
- If we do *not* know t , then there is a problem: we do not know which k to use, so we do not know when to stop doing the Grover iterates. Note that if k gets too big, the success probability $P_k = (\sin((2k+1)\theta))^2$ goes down again! However, a slightly more complicated algorithm due to [62] (basically running the above algorithm with exponentially increasing guesses for k) shows that an *expected* number of $O(\sqrt{N/t})$ queries still suffices to find a solution if there are t solutions. If there is no solution ($t = 0$), then we can easily detect that by checking x_i for the i that the algorithm outputs.
- If we know a lower bound τ on the actual (possibly unknown) number of solutions t , then the above algorithm uses an *expected* number of $O(\sqrt{N/\tau})$ queries. If we run this algorithm for up to three times its expected number of queries, then (by Markov's inequality) with probability at least $2/3$ it will have found a solution. This way we can turn an *expected* runtime into a *worst-case* runtime.
- If we do not know t but would like to reduce the probability of not finding a solution to some small $\varepsilon > 0$, then we can do this using $O(\sqrt{N \log(1/\varepsilon)})$ queries (see Exercise 6).
NB: The important part here is that the $\log(1/\varepsilon)$ is inside the square-root; usual error-reduction by $O(\log(1/\varepsilon))$ repetitions of basic Grover would give the worse upper bound of $O(\sqrt{N} \log(1/\varepsilon))$ queries.

7.3 Amplitude amplification

The analysis that worked for Grover's algorithm is actually much more generally applicable (we will also see it again in the next chapter). In this section we describe a very similar procedure that allows us to amplify the “good” part of the outcome of an algorithm. Quite abstractly, suppose we have a quantum circuit A (without measurements) that acts on m qubits, such that

$$A|0^m\rangle = \sqrt{p}|\psi_1\rangle + \sqrt{1-p}|\psi_0\rangle,$$

where $|\psi_1\rangle$ and $|\psi_0\rangle$ are normalized m -qubit states that are orthogonal to each other (it could for instance be that the last qubit of $|\psi_1\rangle$ is $|1\rangle$ and the last qubit of $|\psi_0\rangle$ is $|0\rangle$). For some reason we like the state $|\psi_1\rangle$ and we want to increase its “weight” $\sqrt{p}$ in the superposition. The following procedure achieves this.

In analogy with the analysis of Grover, think of $|\psi_1\rangle$ as the “good state” and $|\psi_0\rangle$ as the “bad state,” and view these two states as the vertical and horizontal axes in a 2-dimensional picture. Our starting state will be $|U\rangle = A|0^m\rangle$, which plays the role that the uniform state played in Grover, and which is of course easy to obtain by applying A once to basis state $|0^m\rangle$. The angle between $|U\rangle$ and the horizontal axis is $\theta = \arcsin \sqrt{p}$. We would like to rotate this initial state towards the good state, i.e., towards the vertical axis. As for Grover, we could implement the desired rotation as a product of two reflections: a reflection through the bad state and a reflection through $|U\rangle$.

For the first reflection, suppose we have a circuit R_G that can somehow distinguish the good state from the bad state by putting a “−” in front of $|\psi_1\rangle$ and leaving $|\psi_0\rangle$ alone. For example, if the last qubit of $|\psi_1\rangle$ is $|1\rangle$ and the last qubit of $|\psi_0\rangle$ is $|0\rangle$, then R_G would be extremely easy: it would just apply a Z -gate to the last qubit. The second reflection can be implemented as

$$AR_0A^{-1}.$$

It is easy to check that this maps the state $|U\rangle = A|0^m\rangle$ to itself, while every state orthogonal to $|U\rangle$ gets a “−” in front of it, so indeed this reflects through $|U\rangle$. Like before, the product of these two reflections corresponds to a rotation by an angle of 2θ in the 2-dimensional picture.

The following *amplitude amplification* procedure from [66] increases the amplitude of the good state to be close to 1:

1. Setup the starting state $|U\rangle = A|0^m\rangle$
2. Repeat the following $O(1/\sqrt{p})$ times:
 - (a) Reflect through the bad state $|\psi_0\rangle$ (i.e., apply R_G)
 - (b) Reflect through $|U\rangle$ (i.e., apply AR_0A^{-1})

The analysis is the same as for Grover: the initial angle between our algorithm’s state and the horizontal axis is θ and every iteration increases this angle by 2θ , so after k iterations our state is

$$\sin((2k+1)\theta)|\psi_1\rangle + \cos((2k+1)\theta)|\psi_0\rangle.$$

We would like to end up with angle $(2k+1)\theta \approx \pi/2$, because then the amplitude of the good state $|\psi_1\rangle$ would be close to $\sin(\pi/2) = 1$. Hence, like before, we choose k to be $\frac{\pi}{4\theta} - 1/2$ rounded to the nearest integer. This is $O(1/\sqrt{p})$. If we do not know in advance what p is, then we can try out exponentially decreasing guesses for its value, similar to how we handle the case of Grover with unknown number of solutions.

Note that the Hadamard transform $H^{\otimes n}$ can be viewed as an algorithm with success probability $p = t/N$ for a search problem of size N with t solutions, because $H^{\otimes n}|0^n\rangle$ is the uniform superposition over all N locations. Hence Grover’s algorithm is a special case of amplitude amplification, where $m = n$, $A = A^{-1} = H^{\otimes n}$, and R_G corresponds to a phase-query to x .

Amplitude amplification allows to speed up a very large class of classical algorithms: any algorithm A that has some non-trivial probability p of finding a solution to whatever problem we’re trying to solve, can be amplified to success probability nearly 1 by $O(1/\sqrt{p})$ runs of A and A^{-1} provided we can efficiently “recognize” solutions, i.e., implement R_G . In contrast, classically we would need to repeat A $O(1/p)$ times before we have success probability close to 1.

7.4 Application: satisfiability

Grover’s algorithm has many applications: basically any classical algorithm that has some search-component can be improved using Grover’s algorithm as a subroutine. This includes many basic computer applications such as finding shortest paths and minimum spanning trees, various other graph algorithms, etc.

We can also use it to speed up the computation of **NP**-complete problems (see Chapter 13 for the complexity class **NP**), albeit only quadratically, not exponentially. As an example, consider the satisfiability problem: we are given a Boolean formula $\phi(i_1, \dots, i_n)$ and want to know if it has a satisfying assignment, i.e., a setting of the bits $i_1, \dots, i_n$ that makes $\phi(i_1, \dots, i_n) = 1$. A classical brute-force search along all 2^n possible assignments takes time roughly 2^n .

To find a satisfying assignment faster, define the $N = 2^n$ -bit input to Grover’s algorithm by $x_i = \phi(i)$, where $i \in \{0, 1\}^n$. For a given assignment $i = i_1 \dots i_n$ it is easy to compute $\phi(i)$ classically in polynomial time. We can write that computation as a reversible circuit (using only Toffoli gates), corresponding to a unitary U_ϕ that maps $|i, 0, 0\rangle \mapsto |i, \phi(i), w_i\rangle$, where the third register holds some classical workspace the computation may have needed. To apply Grover we need an oracle that puts the answer in the phase and doesn’t leave workspace around (as that could mess up the interference effects, see Exercise 2.10 for an example). Define O_x as the unitary that first applies U_ϕ , then applies a Z -gate to the second register, and then applies U_ϕ^{-1} to “clean up” the workspace again. This has the form we need for Grover: $O_{x,\pm}|i\rangle = (-1)^{x_i}|i\rangle$; here we did not explicitly write the workspace qubits, which start and end in $|0\rangle$. Now we can run Grover and find a satisfying assignment with high probability if there is one, using a number of elementary operations that is $\sqrt{2^n}$ times some polynomial factor.

If brute-force search is basically the best thing we can do classically to solve some particular **NP**-hard problem, then that computation can be sped up quadratically on a quantum computer using Grover search like above. However, there are also **NP**-hard problems where we know algorithms that still run in exponential time, but that are much faster than brute-force search. For example, consider the famous Traveling Salesman Problem (TSP): given an n -vertex graph with weights (distances) on the edges, find the shortest tour in this graph that visits every node exactly once. Since there are $(n-1)!$ many different tours, classical brute-force search would take time $(n-1)!$, times some polynomial in n . Grover’s algorithm could speed this up quadratically. However, there are much more clever classical algorithms for TSP. In particular, the Bellman-Held-Karp dynamic programming algorithm solves TSP in time 2^n , times a polynomial in n . This algorithm is *much* faster than $O(\sqrt{n!})$ (which is roughly $(n/e)^{n/2}$), and is not amenable to a straightforward speed-up using Grover. Nevertheless, it turns out quantum computers can still solve TSP polynomially faster than the best known classical algorithms, albeit in a much more complicated way than by just applying Grover [16].

Exercises

1. (a) Suppose $n = 2$, and $x = x_{00}x_{01}x_{10}x_{11} = 0001$. Give the specific initial state, three intermediate states, and final state in Grover’s algorithm, for $k = 1$ iterations (using the decomposition of one Grover iterate into a product of four unitaries from Eq. (7.1)). What is the success probability?
- (b) Give the final state after $k = 2$ iterations. What is now the success probability?

2. (a) Suppose you have a quantum algorithm for some computational problem that takes $\sqrt{N}$ operations on inputs of size N , each operation of constant cost C . And the best-possible classical algorithm for the same computational problem takes N operations, each of constant cost c . Suppose C is much larger than c (which is certainly the case in the current state of quantum technology: doing one elementary quantum gate is much more expensive than one doing classical logic gate). How large does the input-size N have to be before the quantum algorithm has lower cost than the best-possible classical algorithm?
- (b) Suppose you have a quantum algorithm of cost $C\sqrt{2^n}$ for satisfiability of n -variable Boolean formulas, where the best-possible classical algorithm has cost $c\sqrt{2^n}$, and again C is much larger than c . How large does n have to be before the quantum algorithm has lower cost than the best-possible classical algorithm?
3. Show that if the number of solutions is $t = N/4$, then Grover's algorithm always finds a solution with certainty after just one query. How many queries would a classical algorithm need to find a solution with certainty if $t = N/4$? And if we allow the classical algorithm error probability $1/10$?
4. Suppose we have a string of $N = 2^n$ bits, containing t ones (solutions) and $N - t$ zeroes. You may assume you know the number t .
 - (a) Show that we can use Grover's algorithm to find the positions of *all* t ones, using an expected number of $O(t\sqrt{N})$ queries. You can argue on a high level, no need to draw actual quantum circuits.
 - (b) (H) Show that this can be improved to an expected number of $O(\sqrt{tN})$ queries.
5. At the end of Section 7.2 we claimed without proof that Grover's algorithm can be tweaked to work *with probability 1* if we know the number of solutions exactly. For $N = 2^n$, this question asks you to provide such an exact algorithm for an $x \in \{0, 1\}^N$ with a unique solution (so we are promised that there is exactly one $i \in \{0, 1\}^n$ with $x_i = 1$, and our goal is to find this i).
 - (a) Give the success probability of the basic version of Grover's algorithm after k iterations.
 - (b) Suppose the optimal number of iterations $\tilde{k} = \frac{\pi}{4 \arcsin(1/\sqrt{N})} - \frac{1}{2}$ is not an integer. Show that if we round $\tilde{k}$ up to the nearest integer, doing $\lceil \tilde{k} \rceil$ iterations, then the algorithm will have success probability strictly less than 1.
 - (c) Define a new $2N$ -bit string $y \in \{0, 1\}^{2N}$, indexed by $(n+1)$ -bit strings $j = j_1 \dots j_n j_{n+1}$, by setting

$$y_j = \begin{cases} 1 & \text{if } x_{j_1 \dots j_n} = 1 \text{ and } j_{n+1} = 0, \\ 0 & \text{otherwise.} \end{cases}$$

Show how you can implement the following $(n+1)$ -qubit unitary

$$S_y : |j\rangle \mapsto (-1)^{y_j} |j\rangle,$$

using one query to x (of the usual form $O_x : |i, b\rangle \mapsto |i, b \oplus x_i\rangle$) and a few elementary gates.

- (d) Let $\gamma \in [0, 2\pi)$ and let $U_\gamma = \begin{pmatrix} \cos \gamma & -\sin \gamma \\ \sin \gamma & \cos \gamma \end{pmatrix}$ be the corresponding rotation matrix. Let $\mathcal{A} = H^{\otimes n} \otimes U_\gamma$ be an $(n+1)$ -qubit unitary. What is the probability (as a function of γ) that measuring the state $\mathcal{A}|0^{n+1}\rangle$ in the computational basis gives a solution $j \in \{0, 1\}^{n+1}$ for y (i.e., such that $y_j = 1$)?
- (e) (H) Give a quantum algorithm that finds the unique solution in string x with probability 1 using $O(\sqrt{N})$ queries to x .
6. Given query access to $x \in \{0, 1\}^N$, with unknown Hamming weight $t = |x|$, we want to find a solution, i.e., an index $i \in \{0, \dots, N-1\}$ such that $x_i = 1$. If $x = 0^N$ then our search algorithm should output “no solution.”
- (a) (H) Suppose we know an integer s such that $t \in \{1, \dots, s\}$. Give a quantum algorithm that finds a solution with probability 1, using $O(\sqrt{sN})$ queries to x .
- (b) Suppose we know that $t \in \{s+1, \dots, N\}$. Give a quantum algorithm that finds a solution with probability at least $1 - 2^{-s}$, using $O(\sqrt{sN})$ queries to x .
- (c) For given $\varepsilon > 2^{-N}$, give a quantum algorithm that solves the search problem with probability $\geq 1 - \varepsilon$ using $O(\sqrt{N \log(1/\varepsilon)})$ queries, without assuming anything about t .
7. (H) Here we will *approximately count* the number of 1s in a string $x \in \{0, 1\}^N$. Let $t = |x|$ denote that (unknown) number.
- (a) Given an integer $m \in \{1, \dots, N\}$, describe a quantum algorithm that makes $O(\sqrt{N/m})$ queries to x and decides between the cases $t \leq m/2$ and $t \in [m, 2m]$ with probability at least $2/3$. That is, the algorithm has to output 0 with probability $\geq 2/3$ whenever $t \leq m/2$, has to output 1 with probability $\geq 2/3$ whenever $t \in [m, 2m]$, and can output whatever it wants for other values of t .
- (b) Give a quantum algorithm that uses $O(\sqrt{N} \log \log N)$ queries to x and that outputs an integer m such that, with probability $\geq 2/3$, the unknown t lies between $m/2$ and $2m$.
8. Suppose we have a quantum circuit A acting on m qubits, such that

$$A|0^m\rangle = \sqrt{a}|\phi_1\rangle|1\rangle + \sqrt{1-a}|\phi_0\rangle|0\rangle,$$

where $|\phi_1\rangle$ and $|\phi_0\rangle$ are arbitrary normalized $(m-1)$ -qubit states, and $a \in [0, 1/4]$ is an unknown number. Our goal is to estimate a (this is known as *amplitude estimation*). Let $\mathcal{S}$ be the 2-dimensional subspace spanned by $|\phi_1\rangle|1\rangle$ and $|\phi_0\rangle|0\rangle$.

- (a) Show that in $\mathcal{S}$, the unitary $I \otimes Z$ (where I is the identity on $m-1$ qubits and Z is the phase-flip gate) is a reflection through $|\phi_0\rangle|0\rangle$.
- (b) Let $R_0 = 2|0^m\rangle\langle 0^m| - I$ be a reflection through $|0^m\rangle$. Show that in $\mathcal{S}$, the unitary AR_0A^{-1} is a reflection through $A|0^m\rangle$.
- (c) Show that in $\mathcal{S}$, the unitary $U = AR_0A^{-1} \cdot (I \otimes Z)$ is a rotation over angle 2θ , where $\theta = \arcsin \sqrt{a}$.
- (d) (H) Given some $\varepsilon \in (0, 1/2)$, show how you can use phase estimation (Section 4.6) with $O(1/\varepsilon)$ applications of U to find an approximation $\tilde{a}$ of a such that $|\sqrt{\tilde{a}} - \sqrt{a}| \leq \varepsilon$.

- (e) (H) Suppose we have query-access to a string $x \in \{0, 1\}^N$ of unknown Hamming weight $t = |x|$. Use (d) to compute an integer $\tilde{t}$ such that $|\tilde{t} - t| \leq \sqrt{N}$ with success probability $\geq 2/3$, using $O(\sqrt{N})$ queries to x .
- (f) Suppose $x \in \{0, 1\}^N$ has $|x| \in \{0, 1\}$. Use (d) to compute $|x|$ with success probability $\geq 2/3$, using $O(\sqrt{N})$ queries to x (you may *not* invoke Grover here).
9. Suppose you are given query access to $x \in \{0, 1\}^N$, where $|x| = 1$ and $N = 2^n$. You want to find the unique index $i = i_{n-1} \dots i_0 \in \{0, 1\}^n$ such that $x_i = 1$.
- (a) Let $k \in \{1, \dots, n-1\}$. Fix the first $n-k$ bits of the n -bit index $i_{n-1} \dots i_0$ to specific values $i_{n-1}^* \dots i_k^*$. Give a quantum algorithm to find a solution with success probability 1 (if one exists) among the 2^k indices $i \in \{0, 1\}^n$ that start with $i_{n-1}^* \dots i_k^*$, using $O(\sqrt{2^k})$ queries and $O(\sqrt{2^k} k)$ other gates.
- (b) Give a quantum algorithm that solves the search problem on x using $O(\sqrt{N})$ queries and $O(\sqrt{N} \log \log N)$ other gates.
 Comment: The $O(\sqrt{N} \log \log N)$ gates achieved here is better than standard Grover, which uses $O(\sqrt{N} \log N)$ gates. The $\log \log N$ can be reduced a bit further, to nearly-constant [28].
10. (H) Let $x = x_0 \dots x_{N-1}$ be a sequence of distinct real numbers, where $N = 2^n$, and each x_i can be written exactly using b bits. We can query these in the usual way, i.e., we can apply $(n+b)$ -qubit unitary $O_x : |i, 0^b\rangle \mapsto |i, x_i\rangle$, as well as its inverse. The *minimum* of x is defined as $\min\{x_i \mid i \in \{0, \dots, N-1\}\}$. Give a quantum algorithm that finds (with probability $\geq 2/3$) an index achieving the minimum, using at most $O(\sqrt{N} \log N)$ queries to the input, and prove that this algorithm works.
 Bonus: give a quantum algorithm that uses $O(\sqrt{N})$ queries.
11. Let $x = x_0 \dots x_{N-1}$, where $N = 2^n$ and $x_i \in \{0, 1\}^n$, be an input that we can query in the usual way. We are promised that this input is 2-to-1: for each i there is exactly one other j such that $x_i = x_j$.³ Such an (i, j) -pair is called a *collision*.
- (a) Suppose S is a uniformly randomly chosen set of $s \leq N/2$ elements of $\{0, \dots, N-1\}$. What is the probability that there exists a collision in S ?
- (b) (H) Give a classical randomized algorithm that finds a collision (with probability $\geq 2/3$) using $O(\sqrt{N})$ queries to x .
- (c) (H) Give a quantum algorithm that finds a collision (with probability $\geq 2/3$) using $O(N^{1/3})$ queries.
12. Consider an undirected graph $G = (V, E)$, with vertex set $V = \{1, \dots, n\}$ and edge-set E . We say G is *connected* if, for every pair of vertices $i, j \in V$, there is a path between i and j in the graph. The *adjacency matrix* of G is the $n \times n$ Boolean matrix M where $M_{ij} = 1$ iff $(i, j) \in E$ (note that M is a symmetric matrix because G is undirected). Suppose we are given input graph G in the form of a unitary that allows us to query whether an edge (i, j) is present in G or not:

$$O_M : |i, j, b\rangle \mapsto |i, j, b \oplus M_{ij}\rangle.$$

³The 2-to-1 inputs for Simon's algorithm are a very special case of this, where the collisions are determined by a secret string $s \in \{0, 1\}^n$: x_i equals x_j if $i = j \oplus s$.

- (a) Assume G is connected. Suppose we have a set A of edges which we already know to be in the graph (so $A \subseteq E$; you can think of A as given classically, you don't have to query it). Let $G_A = (V, A)$ be the subgraph induced by only these edges, and suppose G_A is not connected, so it consists of $c > 1$ connected components. Call an edge $(i, j) \in E$ "good" if it connects two of these components. Give a quantum algorithm that finds a good edge with an *expected* number of $O(n/\sqrt{c-1})$ queries to M .
- (b) Give a quantum algorithm that uses at most $O(n^{3/2})$ queries to M and decides (with success probability at least $2/3$) whether G is connected or not.
- (c) Show that classical algorithms for deciding (with success probability at least $2/3$) whether G is connected, need to make $\Omega(n^2)$ queries to M .